Wine Quality Predictor
Project Report of YORAM ACCOH and IVAN KOUASSI

PGE 2 · Machine Learning × Advanced Python

1. Problem Framing

Context.
Wine merchants receive dozens of new bottles every week and must decide which ones deserve the attention of an expert tasting panel. Tasting every wine is time-consuming and expensive. A prediction tool based on chemical measurements can help prioritize the most promising bottles.

Who uses the tool?
A wine merchant or a quality control team. The tool takes the physicochemical measurements of a wine as input and returns a prediction: good (quality ≥ 6) or not good (quality < 6).

What decision does it support?
The merchant uses the prediction to decide which wines to send to the tasting panel first. Wines predicted good are tasted; wines predicted not good are set aside.

Cost of a false positive (FP). The model predicts good but the wine is actually not good. The panel wastes time tasting a mediocre wine and it's moderate cost since no wine is sold, just time is lost.

Cost of a false negative (FN). The model predicts not good but the wine is actually good. A good wine is never tasted and a potential sale is lost; this is a direct financial loss and therefore a higher cost.

Consequence for the metric.
We want to minimize FN more than FP, so we need high recall on good wines. We track F1 (harmonic mean of precision and recall) as our main metric and the confusion matrix to see exactly how many FP and FN the model produces. We do not use accuracy because the classes are slightly imbalanced, a model that always predicts good would get 62.6% accuracy without learning anything.

2. Data

Dataset.
Wine Quality, UCI Machine Learning Repository #186 (Cortez et al., 2009). The dataset comes as two separate CSV files : red wine (1,599 wines) and white wine (4,898 wines) which we merge into a single dataset of 6,497 wines with 11 physicochemical features.
After removing 1,177 exact duplicates, 5,320 wines remain for modeling.

Features.
All 11 features are numerical (float64): fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, alcohol. We also add a categorical feature wine_type (red or white) to let the model distinguish between the two styles.

Target. We create a binary label: good = 1 if quality ≥ 6, else 0. This threshold was chosen because quality 6 is above the dataset mean (5.8) and represents a natural boundary between average and

above-average wines. The resulting classes are: 3,332 good wines (62.6%) and 1,988 not good wines (37.4%) and that is moderately imbalanced.

Key EDA findings.

- No missing values in either file. The dataset is clean.
- Alcohol is the most correlated feature with quality (correlation = 0.47) and with the good label (0.41). Higher quality wines consistently have more alcohol (median alcohol rises from ~10% for quality 5 to ~12% for quality 8.)
- Volatile acidity is negatively correlated with quality (−0.27) means wines with higher acidity tend to be rated lower.
- Free and total sulfur dioxide are strongly correlated with each other (0.72), meaning they carry overlapping information.
- Citric acid and chlorides show near-zero correlation with quality and turn out to be unimportant for the model.

Leakage check. The quality column is dropped before modeling : it was only used to create the binary label and must not be used as a feature.

3. Method

Data split. We perform a single stratified 80/20 train/test split using our `DatasetLoader` class. Stratification ensures that the proportion of good wines is the same in both sets (62.6%). The test set is kept aside and used only once, at the very end.

Pipeline. All preprocessing is inside a scikit-learn Pipeline with a ColumnTransformer: StandardScaler on the 11 numerical features (brings all features to the same scale, which matters for distance-based models and regularized models) and `OneHotEncoder` on wine_type (converts red/white into a numerical indicator). The entire preprocessing is fitted on the training folds only during cross-validation, so nothing leaks from the validation folds.

Baseline.
A `DummyClassifier(strategy="most_frequent")` that always predicts good. F1 = 0.770, ROC-AUC = 0.500 (pure chance). All real models must beat this.

Model comparison. We compare four models under 5-fold stratified cross-validation, scoring on F1 and ROC-AUC:

Model	F1 (CV)	ROC-AUC (CV)
Logistic Regression	0.802	0.804
Linear SVM	0.802	0.804
k-NN	0.799	0.803
Dummy (baseline)	0.770	0.500

All three real models clearly beat the baseline on ROC-AUC, confirming that the chemical measurements contain useful information.

Hyperparameter tuning. We run `GridSearchCV` (5-fold CV, scoring = F1) on all three models. Best results: Logistic Regression C = 0.1 (F1 = 0.803), Linear SVM C = 0.01 (F1 = 0.803), k-NN n_neighbors = 20 (F1 = 0.801). Logistic Regression is selected as the best model though it ties with Linear SVM but is more interpretable.

Threshold selection. We find the best decision threshold using out-of-fold predictions on the training set with `cross_val_predict`, avoiding any use of the test set. We compute the F1 score at every possible threshold using `precision_recall_curve` and pick the threshold that maximises F1.

Best threshold = 0.48.

4. Results

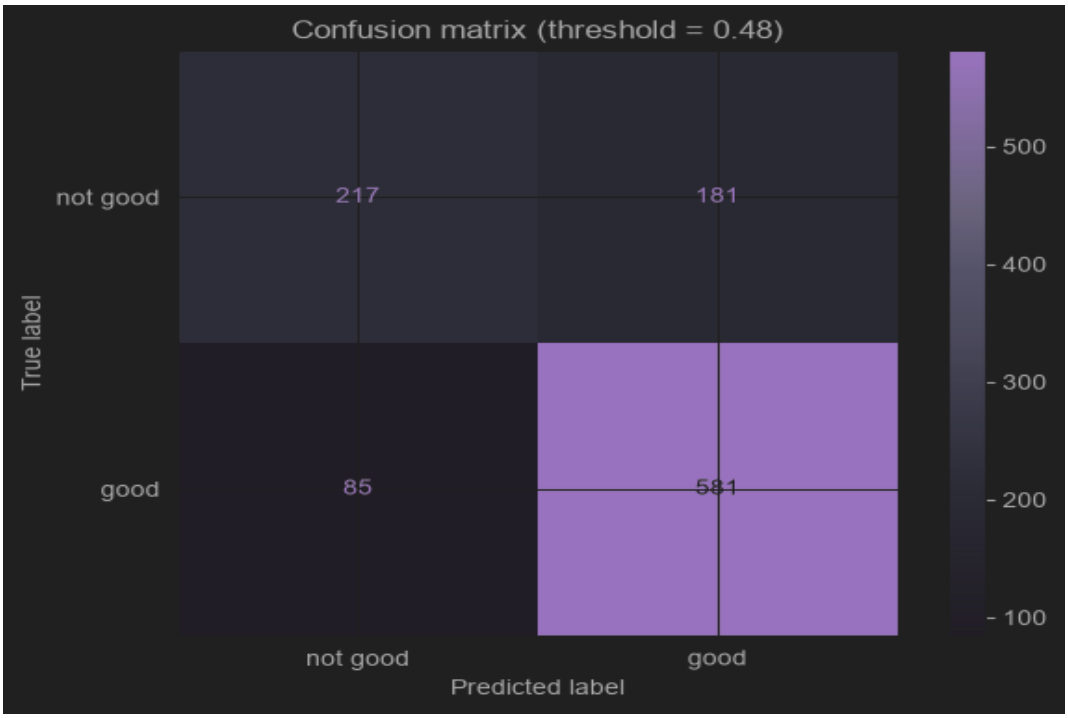
Test set evaluation (used once at the end): F1 = 0.814, ROC-AUC = 0.814.

The test F1 (0.814) is slightly higher than the CV F1 (0.803), which is expected due to natural variation between splits and confirms that the model generalizes well to unseen data.

Confusion matrix (threshold = 0.48):

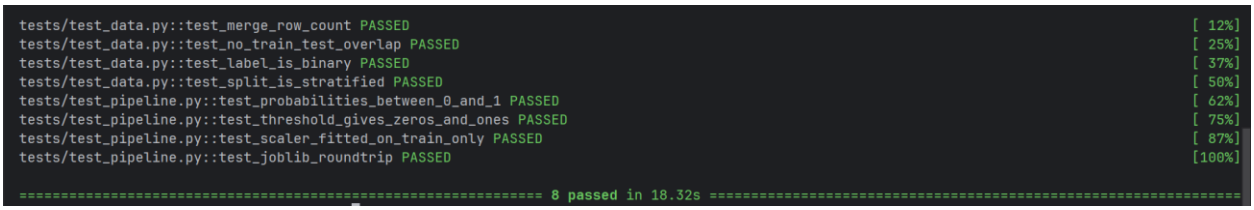
	Predicted not good	Predicted good
Actually not good	217 (TN)	181 (FP)
Actually good	85 (FN)	581 (TP)

The model correctly identifies 581 out of 666 good wines (recall = 87%). It misclassifies 181 bad wines as good (FP) and these are the wines the merchant would waste time tasting. It misses 85 good wines (FN) these are the lost sales.



5. Python Package and Software Engineering”

The project is organized as a reusable Python package with a src/ layout. It includes a CLI with three commands (train, evaluate, predict), model persistence via joblib, and 8 automated pytest tests covering data integrity, pipeline leakage and prediction correctness.”



6. Interpretation and Limits

Most important features. Using permutation importance on the test set:

1. Alcohol (0.065) which is most important by far
2. Volatile acidity (0.051)
3. Residual sugar (0.019)

These features predict quality well, but correlation does not imply causation as in a winemaker cannot just add alcohol to produce a better wine. The model found patterns in this specific dataset, not universal rules of wine quality.

Two limits.

1. The model was trained only on Portuguese Vinho Verde wines rated by one tasting panel. It may not generalize wines from other regions, grape varieties or rating panels.
2. The model is not perfect because 181 bad wines are predicted as good and 85 good wines are missed. With F1 = 0.81, the tool reduces the workload of the tasting panel but does not replace it.

Ethical risks.

The model's definition of good is entirely based on the scores of one tasting panel. A small producer whose wine style differs from what that panel prefers could be unfairly penalized and have their wine systematically rejected. The model gives a probability but no explanation of which features caused the prediction (the merchant cannot tell the producer what to improve.)

7. Future Improvements and Conclusion

Potential extensions include comparing Logistic Regression with non-linear approaches such as Random Forest or Gradient Boosting, evaluating performance across different quality thresholds, and testing the model on an independent external wine dataset. On the software side, adding input validation and clearer error messages to the CLI would make the tool more robust.

The project demonstrates a complete machine learning workflow starting from raw data preparation and feature engineering to model selection, evaluation, packaging, and automated testing. By combining a scikit-learn pipeline with a CLI and a reproducible analysis notebook, the project transforms an experimental model into a reusable Python tool.

References

Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). *Modeling wine preferences by data mining from physicochemical properties.* Decision Support Systems, 47(4), 547–553.

UCI Machine Learning Repository Wine Quality Dataset #186. CC BY 4.0.